

4. Brownout & Traffic Shedding 패턴

1. 마이크로서비스 환경에서의 안정성 확보: 우선순위 기반 부하 분산 기법

마이크로서비스 환경에서 발생하는 예측 불가능한 장애 상황에서도 핵심 서비스의 안정성을 유지하기 위한 '우선순위 기반 부하 분산(Prioritized Load Shedding)' 기법을 소개하고, 이를 효과적으로 구현하기 위한 방안을 제시한다.

1.1. 마이크로서비스 환경의 부하 관리와 장애의 필연성

- 마이크로서비스 환경에서는 개발 속도가 빠르고 서비스 간 의존성이 복잡하여 장애가 필연적으로 발생한다.
- 이러한 장애는 연쇄적인 실패로 이어져 전체 시스템을 마비시킬 수 있다.
- 따라서 예측 불가능한 상황에서도 핵심 사용자 경험을 유지하기 위한 부하 관리 기법이 중요하다.
- 본 보고서는 큐잉 이론과 리틀의 법칙을 기반으로 부하 관리의 기본 원리를 설명하고, 이를 통해 서비스 안정성을 높이는 방법을 다룬다.

1.2. 자동 확장(Auto-Scaling)의 한계점

- 자동 확장은 수요 변화에 대응하고 비용을 최적화하는 데 유용하지만, 몇 가지 한계점을 가진다.
- **느린 응답 속도:** 서비스가 준비되는 데 시간이 걸리는 경우, 자동 확장이 부하 증가를 감지하고 자원을 추가하는 데 시간이 소요되어 사용자에게 지연을 유발할 수 있다.
- **할당량(Quota) 제약:** 자동 확장은 종종 여러 마이크로서비스가 공유하는 컴퓨팅 할당량에 의존하는데, 이는 추가할 수 있는 자원에 제한을 두어 효과를 제한할 수 있다.
- **부하 증폭 및 의존성 문제:** 시스템의 한 부분에 자원을 추가하는 것이 다른 부분을 과부하 상태로 만들거나 연쇄적인 실패를 유발할 수 있다.
 - 포켓몬 GO의 구글 클라우드 로드 밸런서 마이그레이션 사례에서, 로드 밸런싱 계층을 확장했지만 백엔드 스택을 과부하시켜 오히려 장애를 연장시킨 경험이 이를 보여준다.
- 자동 확장은 부하 관리 퍼즐의 한 조각일 뿐, 완전한 해결책은 아니다.

1.3. 동시성 제한(Concurrency Limits)의 이점과 도전 과제

- 동시성 제한은 서비스가 특정 시점에 처리할 수 있는 최대 동시 요청 수를 설정하여 서비스 성능을 최적화한다.
- 이 기법은 서비스가 처리할 수 있는 최대 동시성 한계를 명확히 이해하고 있다는 가정 하에 작동한다.
- **이점:**
 - 최대 동시성 한계 설정으로 인해 초과되는 부하는 차단되어, 서비스는 높은 트래픽 속에서도 최적의 성능을 유지할 수 있다.
 - 이는 서비스의 가용성과 성능을 사용자에게 유지하는 데 도움이 된다.
- **도전 과제:**
 - **최대 동시성 한계 결정의 어려움:** 서비스가 처리할 수 있는 최대 동시 요청 수를 정확히 파악하기 어렵다.
 - 한계를 너무 낮게 설정하면 서비스에 여유가 있어도 요청이 거부될 수 있다.
 - 한계를 너무 높게 설정하면 서비스가 느려지고 응답하지 않게 될 수 있다.
 - **동적인 환경에서의 한계:** 새로운 배포, 의존성의 자동 확장, 머신 구성 변경 등으로 인해 마이크로서비스 환경은 끊임없이 변화하므로, 동시성 한계 값은 빠르게 구식이 될 수 있다.
 - 따라서 동적인 워크로드와 의존성에 반응할 수 있는 동적이고 적응적인 동시성 제어 시스템이 필요하다.

2. Aperture: 지능형 부하 관리를 위한 오픈소스 솔루션

Aperture는 동적인 마이크로서비스 환경에서 지능적인 부하 관리를 가능하게 하는 오픈소스 플랫폼으로, 워크로드 우선순위 지정, 동시성 제한, 분산 속도 제한 등 다양한 기능을 제공한다.

2.1. Aperture의 아키텍처 및 핵심 기능

- **분산 아키텍처:** Aperture는 통합된 관찰 가능성(Observability) 및 제어 계층을 갖춘 분산 아키텍처를 기반으로 한다.
- **Aperture 컨트롤러 (Controller):**
 - 정책이 실행되는 두뇌 역할을 하며, 시스템을 지속적으로 모니터링한다.

- 내장된 텔레메트리 시스템을 통해 서비스 성능, 워크로드, 고객 등급, 요청 유형 등에 대한 메트릭을 수집한다.
- 선언적 정책은 제어 회로(Circuit Graph)로 표현되며, 실시간 메트릭을 분석하여 흐름 제어, 자동 확장 등에 대한 결정을 내린다.
- 정책은 etcd에 저장되며, 쿠버네티스 환경에서는 사용자 정의 리소스 정의(CRD)를 통해 관리할 수 있다.
- **Aperture 에이전트 (Agent):**
 - 서비스 바로 옆에 위치하며, 데몬셋(DaemonSet) 또는 사이드카(Sidecar) 형태로 배포될 수 있다.
 - 워크로드 우선순위 지정을 위한 가중치 기반 공정 큐잉(Weighted Fair Queuing) 스케줄러, 남용 방지를 위한 분산 속도 제한기 등 강력한 흐름 제어 기능을 제공한다.
 - 요청 지연 시간, 오류율, 포화도 등 핵심 신호(Golden Signals)를 OpenTelemetry 기반의 텔레메트리 시스템을 통해 수집한다.
 - HTTP 헤더 또는 SDK를 통해 전달된 속성을 기반으로 요청을 분류하는 고충실도 흐름 분류기(Flow Classifier)를 갖추고 있다.
 - 서비스 메쉬(Istio 등), API 게이트웨이와 통합하거나, Java Spring, Go HTTP 프레임워크와 같은 인기 있는 미들웨어 프레임워크에 코드 변경 없이 통합될 수 있다.
 - 자동 확장, 쿠버네티스와 같은 인프라 API와 연동하여 필요 시 인프라를 확장하는 기능도 제공한다.
- **데이터 저장소:**
 - 정책 저장을 위해 etcd를 사용한다.
 - 에이전트로부터 수집된 텔레메트리 데이터를 저장하고 서비스 수준 목표(SLO) 편차를 지속적으로 모니터링하기 위해 Prometheus를 사용한다.

2.2. Aperture를 활용한 동시성 제한 및 워크로드 우선순위 지정

- Aperture 에이전트는 서비스로 들어오는 각 요청에 대해 허용 여부를 결정하여 서비스의 동시성을 제한한다.
- 서비스는 Aperture 에이전트로부터 받은 '예/아니오' 응답에 따라 요청을 처리하며, 정책에 대한 세부 사항은 추상화되어 있다.

- **동시성 제한:** 토큰 버킷 알고리즘 등을 사용하여 서비스로 허용되는 동시성을 제한함으로써 최적의 성능과 가용성을 보장한다.
- **워크로드 우선순위 지정:**
 - 요청 분류기(Request Classifiers)를 통해 헤더의 속성 등을 기반으로 요청을 분류한다.
 - 각 워크로드의 지연 시간 프로파일을 격리하여 각 요청을 허용하는 비용을 계산한다.
 - 가중치 기반 공정 큐잉 스케줄러는 우선순위가 지정된 방식으로 요청을 스케줄링하여, 어떤 워크로드도 소외되지 않도록 보장한다.
- **실제 적용 사례:**
 - 서비스의 동시성 한계를 10명으로 제한하고, 구독자(Subscribed Users)와 게스트(Guest Users)로 나누어 트래픽을 시뮬레이션한다.
 - 구독자에게 게스트보다 4배 높은 우선순위를 부여하여, 과부하 상태에서도 구독자에게 더 많은 요청이 허용되도록 설정한다.
 - Aperture 정책 적용 전후의 대시보드를 비교하면, 정책 적용 후 응답 시간이 안정적으로 유지되고 구독자 요청이 게스트 요청보다 약 4배 더 많이 수용되는 것을 확인할 수 있다.
 - 이는 Aperture의 동적 동시성 제한 및 워크로드 우선순위 지정 기능이 서비스 성능을 개선하고 장애로부터 보호하는 데 효과적임을 보여준다.

2.3. 포괄적인 부하 관리 전략

- 부하 관리 기법은 반응적(Reactive)에서 사전 예방적(Proactive)으로 이어지는 스펙트럼으로 볼 수 있다.
- **예방적 조치:** 속도 제한, 점진적 기능 출시(Progressive Feature Rollouts) 등은 잠재적인 문제를 미리 방지한다.
- **보호적 조치:** 동적 동시성 제한, 워크로드 우선순위 지정 등은 서비스 상태를 모니터링하여 서비스 중단을 적극적으로 방지한다.
- **반응적 조치:** 자동 확장과 같은 에스컬레이션 및 복구 방법은 문제가 발생한 후에 해결한다.
- 이러한 기법들을 조합하여 변화하는 워크로드와 의존성에 적응하면서 높은 가용성, 확장성, 신뢰성을 보장하는 포괄적인 부하 관리 전략을 수립할 수 있다.

- 이러한 전략을 효과적으로 구현하기 위해서는 중앙 집중식 인식과 모든 기법 간의 조율이 필요하며, 이는 제어 시스템 문제이자 Aperture의 회로 기반 접근 방식과 관련이 있다.